쉬운 **PyTorch v1.0**

입문자를 위한 필수 가이드

저자*: jw*

이 책은 PyTorch 의 핵심 함수들을 카테고리별로 정리했습니다. 각 함수마다 다음 내용을 포함합니다:

- 코드 예제 - 실행 가능한 Python 코드
- 실행 결과 - 실제 출력 결과
- 설명 - 간단명료한 설명

PyTorch 2.8.0 기반

목차

0. 헬로월드
1. 텐서 생성
2. 기본 연산
3. 형태 변환
4. 인덱싱과 슬라이싱
5. 축소 연산
6. 수학 함수
7. 선형대수
8. 신경망 함수
9. 손실 함수
10. 풀링과 컨볼루션
11. 고급 연산
12. 자동미분
13. 디바이스 연산
14. 유틸리티
15. 비교 연산
16. 텐서 메서드

헬로월드

이 장에서는 완전한 신경망을 하나의 간단한 예제로 보여드립니다. 작은 신경망에게 XOR 함수를 가르쳐 봅시다.

XOR 이란**?**

입력 A	입력 B	출력 (A XOR B)
0	0	0
0	1	1
1	0	1
1	1	0

4 단계 과정

1. 데이터 - 입력/출력 쌍 정의
2. 모델 - 신경망 생성

3. 학습 - 데이터로부터 배우기 (순전파 → 손실 → 역전파 → 갱신)
4. 추론 - 예측하기

전체 코드

```
import torch
import torch.nn as nn

# === 헬로월드: 초간단 신경망 ===
# 목표: XOR 함수 학습 (0^0=0, 0^1=1, 1^0=1, 1^1=0)

# 1. 데이터 - 입력/출력 쌍
X = torch.tensor([[0., 0.],
                  [0., 1.],
                  [1., 0.],
                  [1., 1.]])

y = torch.tensor([[0.],
                  [1.],
                  [1.],
                  [0.]])

print("=== 데이터 ===")
print(f" 입력 X:\n{X}")
print(f" 정답 y: {y.flatten().tolist()}")

# 2. 모델 - 2 층 신경망
model = nn.Sequential(
    nn.Linear(2, 4),   # 입력 2 개 -> 은닉 4 개
    nn.ReLU(),         # 활성화 함수
    nn.Linear(4, 1),   # 은닉 4 개 -> 출력 1 개
    nn.Sigmoid()       # 출력 0~1
)

print(f"\n=== 모델 ===")
print(model)

# 3. 학습
criterion = nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=1.0)

print(f"\n=== 학습 ===")
for epoch in range(1000):
    pred = model(X)
    loss = criterion(pred, y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    if epoch % 200 == 0:
        print(f" 에폭 {epoch:4d}, 손실: {loss.item():.4f}")

# 4. 추론
print(f"\n=== 추론 ===")
```

```python
with torch.no_grad():
    predictions = model(X)
    rounded = (predictions > 0.5).float()

print(" 입력 -> 예측 -> 반올림 -> 정답")
for i in range(4):
    inp = X[i].tolist()
    pred_val = predictions[i].item()
    round_val = int(rounded[i].item())
    target = int(y[i].item())
    status = " 정답" if round_val == target else " 오답"
    print(f"{inp} -> {pred_val:.3f} -> {round_val} -> {target} {status}")

accuracy = (rounded == y).float().mean()
print(f"\n정확도: {accuracy.item()*100:.0f}%")
```

실행 결과

```
=== 데이터 ===
입력 X:
tensor([[0., 0.],
        [0., 1.],
        [1., 0.],
        [1., 1.]])
정답 y: [0.0, 1.0, 1.0, 0.0]

=== 모델 ===
Sequential(
  (0): Linear(in_features=2, out_features=4, bias=True)
  (1): ReLU()
  (2): Linear(in_features=4, out_features=1, bias=True)
  (3): Sigmoid()
)

=== 학습 ===
에폭    0, 손실: 0.2455
에폭  200, 손실: 0.1671
에폭  400, 손실: 0.1668
에폭  600, 손실: 0.1668
에폭  800, 손실: 0.1667

=== 추론 ===
입력 -> 예측 -> 반올림 -> 정답
[0.0, 0.0] -> 0.334 -> 0 -> 0 정답
[0.0, 1.0] -> 0.988 -> 1 -> 1 정답
[1.0, 0.0] -> 0.334 -> 0 -> 1 오답
[1.0, 1.0] -> 0.334 -> 0 -> 0 정답

정확도: 75%
```

핵심 개념

- `nn.Sequential` - 레이어를 순차적으로 쌓기

- `nn.Linear(in, out)` - 완전연결층 (Fully Connected Layer)
- `nn.ReLU()` - 활성화 함수
- `nn.MSELoss()` - 손실 함수 (얼마나 틀렸나?)
- `optimizer.zero_grad()` - 그래디언트 초기화
- `loss.backward()` - 그래디언트 계산 (역전파)
- `optimizer.step()` - 가중치 업데이트
- `torch.no_grad()` - 추론시 그래디언트 비활성화

끝! 첫 번째 신경망 학습을 완료했습니다.

텐서 생성

`torch.tensor(data)` 데이터 *(리스트, numpy 배열 등)* 로 텐서를 생성합니다. *dtype* 을 자동으로 추론합니다.

예제:

```
import torch
# Create tensor from Python list
x = torch.tensor([[1, 2], [3, 4]])
print("Tensor from list:")
print(x)
print(f"Shape: {x.shape}, dtype: {x.dtype}")
```

실행 결과:

```
Tensor from list:
tensor([[1, 2],
        [3, 4]])
Shape: torch.Size([2, 2]), dtype: torch.int64
```

`torch.zeros(*size)` *0* 으로 채워진 텐서를 생성합니다. 초기화에 유용합니다.

예제:

```
import torch
# Create tensor filled with zeros
x = torch.zeros(2, 3)
print("Zeros tensor (2x3):")
print(x)
```

실행 결과:

```
Zeros tensor (2x3):
tensor([[0., 0., 0.],
        [0., 0., 0.]])
```

`torch.ones(*size)` *1* 로 채워진 텐서를 생성합니다. 마스크나 초기화에 자주 사용됩니다.

예제:

```
import torch
# Create tensor filled with ones
x = torch.ones(3, 2)
print("Ones tensor (3x2):")
print(x)
```

실행 결과:

```
Ones tensor (3x2):
tensor([[1., 1.],
        [1., 1.],
        [1., 1.]])
```

`torch.eye(n)  # identity matrix` 단위 행렬을 생성합니다 *(*대각선은 *1,* 나머지는 *0).* 선형대수에서 사용됩니다*.*

예제:

```python
import torch
# Create identity matrix
x = torch.eye(3)
print("Identity matrix (3x3):")
print(x)
```

실행 결과:

```
Identity matrix (3x3):
tensor([[1., 0., 0.],
        [0., 1., 0.],
        [0., 0., 1.]])
```

`torch.arange(start, end, step)` 균등 간격의 *1D* 텐서를 생성합니다*. Python* 의 *range()* 와 유사합니다*.*

예제:

```python
import torch
# Create range tensor
x = torch.arange(0, 10, 2)
print("Range [0, 10) step 2:")
print(x)
```

실행 결과:

```
Range [0, 10) step 2:
tensor([0, 2, 4, 6, 8])
```

`torch.linspace(start, end, steps)` 시작과 끝 사이에 균등하게 분배된 점들로 텐서를 생성합니다*.*

예제:

```python
import torch
# Create linearly spaced tensor
x = torch.linspace(0, 1, 5)
print("Linspace 0 to 1, 5 points:")
print(x)
```

실행 결과:

```
Linspace 0 to 1, 5 points:
tensor([0.0000, 0.2500, 0.5000, 0.7500, 1.0000])
```

torch.logspace(start, end, steps) 로그 스케일로 균등 분배된 텐서를 생성합니다. 학습률 설정에 유용합니다.

예제:

```
import torch
# Create logarithmically spaced tensor
x = torch.logspace(0, 2, 3)  # 10^0, 10^1, 10^2
print("Logspace 10^0 to 10^2:")
print(x)
```

실행 결과:

```
Logspace 10^0 to 10^2:
tensor([  1.,  10., 100.])
```

torch.rand(*size) # uniform [0,1) *0* 과 *1* 사이의 균등 분포 난수 텐서를 생성합니다.

예제:

```
import torch
torch.manual_seed(42)
# Create random tensor [0, 1)
x = torch.rand(2, 3)
print("Random uniform [0,1):")
print(x)
```

실행 결과:

```
Random uniform [0,1):
tensor([[0.8823, 0.9150, 0.3829],
        [0.9593, 0.3904, 0.6009]])
```

torch.randn(*size) # normal N(0,1) 표준 정규 분포 *(평균 =0, 표준편차 =1)* 에서 샘플링한 텐서를 생성합니다.

예제:

```
import torch
torch.manual_seed(42)
# Create random tensor from normal distribution
x = torch.randn(2, 3)
print("Random normal N(0,1):")
print(x)
```

실행 결과:

```
Random normal N(0,1):
tensor([[ 0.3367,  0.1288,  0.2345],
        [ 0.2303, -1.1229, -0.1863]])
```

torch.randint(low, high, size) *[low, high)* 범위의 랜덤 정수 텐서를 생성합니다.

예제:

```
import torch
torch.manual_seed(42)
# Create random integers
x = torch.randint(0, 10, (2, 3))
```

```
print("Random integers [0, 10):")
print(x)
```

실행 결과:

```
Random integers [0, 10):
tensor([[2, 7, 6],
        [4, 6, 5]])
```

torch.empty(*size) 초기화되지 않은 텐서를 생성합니다. *zeros/ones* 보다 빠르지만 쓰레기 값을 포함합니다.

예제:

```
import torch
# Create uninitialized tensor
x = torch.empty(2, 2)
print("Empty tensor (uninitialized):")
print(x)
print("Warning: Contains garbage values!")
```

실행 결과:

```
Empty tensor (uninitialized):
tensor([[0., 0.],
        [0., 0.]])
Warning: Contains garbage values!
```

torch.full(size, fill_value) 특정 값으로 채워진 텐서를 생성합니다. 상수에 유용합니다.

예제:

```
import torch
# Create tensor filled with specific value
x = torch.full((2, 3), 7.0)
print("Tensor filled with 7.0:")
print(x)
```

실행 결과:

```
Tensor filled with 7.0:
tensor([[7., 7., 7.],
        [7., 7., 7.]])
```

torch.zeros_like(x), ones_like(x) 입력 텐서와 같은 형태와 *dtype* 의 *0/1* 텐서를 생성합니다.

예제:

```
import torch
original = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
zeros = torch.zeros_like(original)
ones = torch.ones_like(original)
print("Original:", original.shape)
print("Zeros like:", zeros)
print("Ones like:", ones)
```

실행 결과:

```
Original: torch.Size([2, 2])
Zeros like: tensor([[0., 0.],
        [0., 0.]])
Ones like: tensor([[1., 1.],
        [1., 1.]])
```

기본 연산

torch.add(a, b) or a + b 요소별 덧셈입니다. 다른 형태에 대해 브로드캐스팅을 지원합니다.

예제:

```python
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([4, 5, 6])
result = torch.add(a, b)
print(f"{a} + {b} = {result}")
```

실행 결과:

```
tensor([1, 2, 3]) + tensor([4, 5, 6]) = tensor([5, 7, 9])
```

torch.sub(a, b) or a - b 요소별 뺄셈입니다. 브로드캐스팅을 지원합니다.

예제:

```python
import torch
a = torch.tensor([5, 6, 7])
b = torch.tensor([1, 2, 3])
result = torch.sub(a, b)
print(f"{a} - {b} = {result}")
```

실행 결과:

```
tensor([5, 6, 7]) - tensor([1, 2, 3]) = tensor([4, 4, 4])
```

torch.mul(a, b) or a * b 요소별 곱셈 *(*하다마드 곱*)* 입니다. 행렬 곱셈이 아닙니다.

예제:

```python
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([4, 5, 6])
result = torch.mul(a, b)
print(f"{a} * {b} = {result}")
```

실행 결과:

```
tensor([1, 2, 3]) * tensor([4, 5, 6]) = tensor([ 4, 10, 18])
```

torch.div(a, b) or a / b 요소별 나눗셈입니다. 정수 나눗셈을 피하려면 *float* 텐서를 사용하세요.

예제:

```python
import torch
a = torch.tensor([10.0, 20.0, 30.0])
b = torch.tensor([2.0, 4.0, 5.0])
```

```
result = torch.div(a, b)
print(f"{a} / {b} = {result}")
```

실행 결과:

```
tensor([10., 20., 30.]) / tensor([2., 4., 5.]) = tensor([5., 5., 6.])
```

`torch.matmul(a, b) or a @ b` 행렬 곱셈입니다. *2D* 텐서의 경우 표준 행렬 곱을 수행합니다.

예제:

```
import torch
a = torch.tensor([[1, 2], [3, 4]])
b = torch.tensor([[5, 6], [7, 8]])
result = torch.matmul(a, b)
print("Matrix A:")
print(a)
print("Matrix B:")
print(b)
print("A @ B:")
print(result)
```

실행 결과:

```
Matrix A:
tensor([[1, 2],
        [3, 4]])
Matrix B:
tensor([[5, 6],
        [7, 8]])
A @ B:
tensor([[19, 22],
        [43, 50]])
```

`torch.pow(a, exp) or a ** exp` 요소별 거듭제곱 연산입니다. 스칼라 또는 텐서 지수를 사용할 수 있습니다.

예제:

```
import torch
x = torch.tensor([1.0, 2.0, 3.0])
result = torch.pow(x, 2)
print(f"{x} ** 2 = {result}")
```

실행 결과:

```
tensor([1., 2., 3.]) ** 2 = tensor([1., 4., 9.])
```

`torch.abs(x)  # absolute value` 각 요소의 절대값을 반환합니다.

예제:

```
import torch
x = torch.tensor([-1, -2, 3, -4])
result = torch.abs(x)
print(f"abs({x}) = {result}")
```

실행 결과:

```
abs(tensor([-1, -2,  3, -4])) = tensor([1, 2, 3, 4])
```

`torch.neg(x)` **`# negative`** 각 요소의 음수를 반환합니다. *-x* 와 동일합니다.

예제:

```
import torch
x = torch.tensor([1, -2, 3])
result = torch.neg(x)
print(f"neg({x}) = {result}")
```

실행 결과:

```
neg(tensor([ 1, -2,  3])) = tensor([-1,  2, -3])
```

`torch.reciprocal(x)` **`# 1/x`** 각 요소의 역수 *(1/x)* 를 반환합니다.

예제:

```
import torch
x = torch.tensor([1.0, 2.0, 4.0])
result = torch.reciprocal(x)
print(f"1/{x} = {result}")
```

실행 결과:

```
1/tensor([1., 2., 4.]) = tensor([1.0000, 0.5000, 0.2500])
```

`torch.remainder(a, b)` **`# remainder`** 요소별 나머지 *(*모듈로 연산*)* 입니다.

예제:

```
import torch
a = torch.tensor([10, 11, 12])
b = torch.tensor([3, 3, 3])
result = torch.remainder(a, b)
print(f"{a} % {b} = {result}")
```

실행 결과:

```
tensor([10, 11, 12]) % tensor([3, 3, 3]) = tensor([1, 2, 0])
```

▯ 형태 변환

`x.reshape(*shape)` 새로운 형태의 텐서를 반환합니다. 전체 요소 수가 일치해야 합니다. 데이터를 복사할 수 있습니다.

예제:

```
import torch
x = torch.arange(6)
print(f"Original: {x}")
reshaped = x.reshape(2, 3)
print("Reshaped to (2, 3):")
print(reshaped)
```

실행 결과:

```
Original: tensor([0, 1, 2, 3, 4, 5])
Reshaped to (2, 3):
tensor([[0, 1, 2],
        [3, 4, 5]])
```

`x.view(*shape)` 새로운 형태의 뷰를 반환합니다. 연속 메모리가 필요합니다. 데이터를 공유합니다.

예제:

```python
import torch
x = torch.arange(12)
print(f"Original: {x}")
viewed = x.view(3, 4)
print("View as (3, 4):")
print(viewed)
```

실행 결과:

```
Original: tensor([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11])
View as (3, 4):
tensor([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]])
```

`x.transpose(dim0, dim1)` 두 차원을 교환합니다. *2D* 의 경우 행렬 전치와 동일합니다.

예제:

```python
import torch
x = torch.tensor([[1, 2, 3], [4, 5, 6]])
print("Original (2x3):")
print(x)
transposed = x.transpose(0, 1)
print("Transposed (3x2):")
print(transposed)
```

실행 결과:

```
Original (2x3):
tensor([[1, 2, 3],
        [4, 5, 6]])
Transposed (3x2):
tensor([[1, 4],
        [2, 5],
        [3, 6]])
```

`x.permute(*dims)` 모든 차원을 재배열합니다. *transpose* 보다 유연합니다.

예제:

```python
import torch
x = torch.randn(2, 3, 4)
print(f"Original shape: {x.shape}")
permuted = x.permute(2, 0, 1)
print(f"Permuted shape: {permuted.shape}")
```

실행 결과:

```
Original shape: torch.Size([2, 3, 4])
Permuted shape: torch.Size([4, 2, 3])
```

x.squeeze(dim) 크기가 *1* 인 차원을 제거합니다. 텐서 랭크를 줄입니다.

예제:

```
import torch
x = torch.zeros(1, 3, 1, 4)
print(f"Original shape: {x.shape}")
squeezed = x.squeeze()
print(f"Squeezed shape: {squeezed.shape}")
```

실행 결과:

```
Original shape: torch.Size([1, 3, 1, 4])
Squeezed shape: torch.Size([3, 4])
```

x.unsqueeze(dim) 지정된 위치에 크기 *1* 의 차원을 추가합니다.

예제:

```
import torch
x = torch.tensor([1, 2, 3])
print(f"Original shape: {x.shape}")
unsqueezed = x.unsqueeze(0)
print(f"Unsqueezed at dim 0: {unsqueezed.shape}")
print(unsqueezed)
```

실행 결과:

```
Original shape: torch.Size([3])
Unsqueezed at dim 0: torch.Size([1, 3])
tensor([[1, 2, 3]])
```

x.flatten(start_dim, end_dim) 텐서를 *1D* 로 평탄화합니다. 부분 평탄화를 위한 선택적 시작/끝 차원이 있습니다.

예제:

```
import torch
x = torch.randn(2, 3, 4)
print(f"Original shape: {x.shape}")
flat = x.flatten()
print(f"Flattened: {flat.shape}")
```

실행 결과:

```
Original shape: torch.Size([2, 3, 4])
Flattened: torch.Size([24])
```

x.expand(*sizes) 크기가 *1* 인 차원을 따라 반복하여 텐서를 확장합니다. 데이터를 복사하지 않습니다.

예제:

```
import torch
x = torch.tensor([[1], [2], [3]])
print(f"Original: {x.shape}")
```

```
expanded = x.expand(3, 4)
print("Expanded to (3, 4):")
print(expanded)
```

실행 결과:

```
Original: torch.Size([3, 1])
Expanded to (3, 4):
tensor([[1, 1, 1, 1],
        [2, 2, 2, 2],
        [3, 3, 3, 3]])
```

`x.repeat(*sizes)` 각 차원을 따라 텐서를 반복합니다. 새로운 메모리를 생성합니다.

예제:

```
import torch
x = torch.tensor([1, 2])
print(f"Original: {x}")
repeated = x.repeat(3)
print(f"Repeated 3x: {repeated}")
```

실행 결과:

```
Original: tensor([1, 2])
Repeated 3x: tensor([1, 2, 1, 2, 1, 2])
```

`x.contiguous()` 메모리에서 연속된 텐서를 반환합니다. *transpose* 후 *view()* 전에 필요합니다.

예제:

```
import torch
x = torch.randn(3, 4)
y = x.transpose(0, 1)
print(f"Is contiguous: {y.is_contiguous()}")
z = y.contiguous()
print(f"After contiguous(): {z.is_contiguous()}")
```

실행 결과:

```
Is contiguous: False
After contiguous(): True
```

인덱싱과 슬라이싱

`x[i] # index` 기본 인덱싱으로 인덱스 *i* 의 행/요소를 반환합니다.

예제:

```
import torch
x = torch.tensor([[1, 2, 3], [4, 5, 6]])
print("Tensor:")
print(x)
print(f"x[0] = {x[0]}")
print(f"x[1] = {x[1]}")
```

실행 결과:

```
Tensor:
tensor([[1, 2, 3],
        [4, 5, 6]])
x[0] = tensor([1, 2, 3])
x[1] = tensor([4, 5, 6])
```

`x[i:j]  # slice` *start:stop:step* 으로 슬라이싱합니다. *Python* 리스트처럼 작동합니다.

예제:

```python
import torch
x = torch.arange(10)
print(f"Original: {x}")
print(f"x[2:5] = {x[2:5]}")
print(f"x[::2] = {x[::2]}")
```

실행 결과:

```
Original: tensor([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
x[2:5] = tensor([2, 3, 4])
x[::2] = tensor([0, 2, 4, 6, 8])
```

`x[..., i]  # ellipsis` 줄임표 *(...)* 는 나머지 모든 차원을 나타냅니다.

예제:

```python
import torch
x = torch.randn(2, 3, 4)
print(f"Shape: {x.shape}")
print(f"x[..., 0] shape: {x[..., 0].shape}")
print(f"x[0, ...] shape: {x[0, ...].shape}")
```

실행 결과:

```
Shape: torch.Size([2, 3, 4])
x[..., 0] shape: torch.Size([2, 3])
x[0, ...] shape: torch.Size([3, 4])
```

`x[:, -1]  # last column` 음수 인덱싱은 끝에서부터 작동합니다. *-1* 은 마지막 요소입니다.

예제:

```python
import torch
x = torch.tensor([[1, 2, 3], [4, 5, 6]])
print("Tensor:")
print(x)
print(f"Last column x[:, -1] = {x[:, -1]}")
```

실행 결과:

```
Tensor:
tensor([[1, 2, 3],
        [4, 5, 6]])
Last column x[:, -1] = tensor([3, 6])
```

`torch.index_select(x, dim, idx)` 인덱스 텐서를 사용하여 차원을 따라 요소를 선택합니다.

예제:

```
import torch
x = torch.tensor([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
idx = torch.tensor([0, 2])
result = torch.index_select(x, 0, idx)
print("Original:")
print(x)
print(f"Select rows {idx.tolist()}:")
print(result)
```

실행 결과:

```
Original:
tensor([[1, 2, 3],
        [4, 5, 6],
        [7, 8, 9]])
Select rows [0, 2]:
tensor([[1, 2, 3],
        [7, 8, 9]])
```

`torch.masked_select(x, mask)` 마스크가 *True* 인 요소의 *1D* 텐서를 반환합니다.

예제:

```
import torch
x = torch.tensor([[1, 2], [3, 4]])
mask = x > 2
print(f"Tensor: {x}")
print(f"Mask (>2): {mask}")
print(f"Selected: {torch.masked_select(x, mask)}")
```

실행 결과:

```
Tensor: tensor([[1, 2],
        [3, 4]])
Mask (>2): tensor([[False, False],
        [ True,  True]])
Selected: tensor([3, 4])
```

`torch.gather(x, dim, idx)  # gather` 인덱스에 따라 축을 따라 값을 수집합니다. 분포에서 선택할 때 유용합니다.

예제:

```
import torch
x = torch.tensor([[1, 2], [3, 4]])
idx = torch.tensor([[0, 0], [1, 0]])
result = torch.gather(x, 1, idx)
print("Original:")
print(x)
print(f"Gather with indices {idx.tolist()}:")
print(result)
```

실행 결과:

```
Original:
tensor([[1, 2],
        [3, 4]])
Gather with indices [[0, 0], [1, 0]]:
tensor([[1, 1],
        [4, 3]])
```

`torch.scatter(x, dim, idx, src)` *idx* 로 지정된 위치에 *src* 의 값을 *x* 에 씁니다. *gather* 의 역연산입니다.

예제:

```
import torch
x = torch.zeros(3, 5)
idx = torch.tensor([[0, 1, 2, 0, 0], [2, 0, 0, 1, 2]])
src = torch.ones(2, 5)
result = x.scatter(0, idx, src)
print("Scatter result:")
print(result)
```

실행 결과:

```
Scatter result:
tensor([[1., 1., 1., 1., 1.],
        [0., 1., 0., 1., 0.],
        [1., 0., 1., 0., 1.]])
```

`torch.where(cond, x, y)  # conditional` 조건이 *True* 인 곳에서는 *x* 의 요소를, 그렇지 않으면 *y* 의 요소를 반환합니다.

예제:

```
import torch
x = torch.tensor([1, 2, 3, 4, 5])
y = torch.tensor([10, 20, 30, 40, 50])
cond = x > 3
result = torch.where(cond, x, y)
print(f"x: {x}")
print(f"y: {y}")
print(f"where(x>3, x, y): {result}")
```

실행 결과:

```
x: tensor([1, 2, 3, 4, 5])
y: tensor([10, 20, 30, 40, 50])
where(x>3, x, y): tensor([10, 20, 30,  4,  5])
```

`torch.take(x, indices)  # flat index` 텐서를 *1D* 로 취급하고 주어진 인덱스의 요소를 반환합니다.

예제:

```
import torch
x = torch.tensor([[1, 2, 3], [4, 5, 6]])
idx = torch.tensor([0, 2, 5])
result = torch.take(x, idx)
```

```
print(f"Tensor (flattened would be {x.flatten().tolist()})")
print(f"Take indices {idx.tolist()}: {result}")
```

실행 결과:

```
Tensor (flattened would be [1, 2, 3, 4, 5, 6])
Take indices [0, 2, 5]: tensor([1, 3, 6])
```

Σ 축소 연산

`x.sum(dim, keepdim)` 요소를 합산합니다. 선택적 *dim* 은 축을 지정합니다. *keepdim* 은 차원을 유지합니다.

예제:

```
import torch
x = torch.tensor([[1, 2, 3], [4, 5, 6]])
print("Tensor:")
print(x)
print(f"Sum all: {x.sum()}")
print(f"Sum dim=0: {x.sum(dim=0)}")
print(f"Sum dim=1: {x.sum(dim=1)}")
```

실행 결과:

```
Tensor:
tensor([[1, 2, 3],
        [4, 5, 6]])
Sum all: 21
Sum dim=0: tensor([5, 7, 9])
Sum dim=1: tensor([ 6, 15])
```

`x.mean(dim, keepdim)` 평균을 계산합니다. *float* 텐서가 필요합니다.

예제:

```
import torch
x = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
print("Tensor:")
print(x)
print(f"Mean all: {x.mean()}")
print(f"Mean dim=1: {x.mean(dim=1)}")
```

실행 결과:

```
Tensor:
tensor([[1., 2.],
        [3., 4.]])
Mean all: 2.5
Mean dim=1: tensor([1.5000, 3.5000])
```

`x.std(dim, unbiased)` 표준편차를 계산합니다. *unbiased=True* 는 *N-1* 분모를 사용합니다.

예제:

```python
import torch
x = torch.tensor([1.0, 2.0, 3.0, 4.0, 5.0])
print(f"Data: {x}")
print(f"Std (unbiased): {x.std():.4f}")
print(f"Std (biased): {x.std(unbiased=False):.4f}")
```

실행 결과:

```
Data: tensor([1., 2., 3., 4., 5.])
Std (unbiased): 1.5811
Std (biased): 1.4142
```

`x.var(dim, unbiased)` 분산 *(표준편차의 제곱)* 을 계산합니다.

예제:

```python
import torch
x = torch.tensor([1.0, 2.0, 3.0, 4.0, 5.0])
print(f"Data: {x}")
print(f"Variance: {x.var():.4f}")
```

실행 결과:

```
Data: tensor([1., 2., 3., 4., 5.])
Variance: 2.5000
```

`x.max(dim)  # values & indices` 차원을 따라 최대값과 인덱스를 반환합니다.

예제:

```python
import torch
x = torch.tensor([[1, 5, 3], [4, 2, 6]])
print("Tensor:")
print(x)
vals, idxs = x.max(dim=1)
print(f"Max per row: values={vals}, indices={idxs}")
```

실행 결과:

```
Tensor:
tensor([[1, 5, 3],
        [4, 2, 6]])
Max per row: values=tensor([5, 6]), indices=tensor([1, 2])
```

`x.min(dim)  # values & indices` 차원을 따라 최소값과 인덱스를 반환합니다.

예제:

```python
import torch
x = torch.tensor([[1, 5, 3], [4, 2, 6]])
print("Tensor:")
print(x)
vals, idxs = x.min(dim=1)
print(f"Min per row: values={vals}, indices={idxs}")
```

실행 결과:

```
Tensor:
tensor([[1, 5, 3],
        [4, 2, 6]])
Min per row: values=tensor([1, 2]), indices=tensor([0, 1])
```

`x.argmax(dim)  # indices only` 최대값의 인덱스를 반환합니다. *softmax* 출력과 자주 사용됩니다.

예제:

```
import torch
x = torch.tensor([[1, 5, 3], [4, 2, 6]])
print("Tensor:")
print(x)
print(f"Argmax (all): {x.argmax()}")
print(f"Argmax dim=1: {x.argmax(dim=1)}")
```

실행 결과:

```
Tensor:
tensor([[1, 5, 3],
        [4, 2, 6]])
Argmax (all): 5
Argmax dim=1: tensor([1, 2])
```

`x.argmin(dim)  # indices only` 최소값의 인덱스를 반환합니다.

예제:

```
import torch
x = torch.tensor([[1, 5, 3], [4, 2, 6]])
print("Tensor:")
print(x)
print(f"Argmin dim=1: {x.argmin(dim=1)}")
```

실행 결과:

```
Tensor:
tensor([[1, 5, 3],
        [4, 2, 6]])
Argmin dim=1: tensor([0, 1])
```

`x.median(dim)` 차원을 따라 중앙값과 인덱스를 반환합니다.

예제:

```
import torch
x = torch.tensor([[1, 5, 3], [4, 2, 6]])
print("Tensor:")
print(x)
vals, idxs = x.median(dim=1)
print(f"Median per row: values={vals}, indices={idxs}")
```

실행 결과:

```
Tensor:
tensor([[1, 5, 3],
        [4, 2, 6]])
Median per row: values=tensor([3, 4]), indices=tensor([2, 0])
```

x.mode(dim) 차원을 따라 가장 빈번한 값을 반환합니다.

예제:

```
import torch
x = torch.tensor([[1, 1, 2], [3, 3, 3]])
print("Tensor:")
print(x)
vals, idxs = x.mode(dim=1)
print(f"Mode per row: values={vals}")
```

실행 결과:

```
Tensor:
tensor([[1, 1, 2],
        [3, 3, 3]])
Mode per row: values=tensor([1, 3])
```

x.prod(dim) **# product** 차원을 따라 요소들의 곱을 계산합니다.

예제:

```
import torch
x = torch.tensor([[1, 2, 3], [4, 5, 6]])
print("Tensor:")
print(x)
print(f"Product all: {x.prod()}")
print(f"Product dim=1: {x.prod(dim=1)}")
```

실행 결과:

```
Tensor:
tensor([[1, 2, 3],
        [4, 5, 6]])
Product all: 720
Product dim=1: tensor([  6, 120])
```

x.cumsum(dim) **# cumulative sum** 차원을 따라 누적 합을 계산합니다. 각 요소는 이전 모든 요소의 합입니다.

예제:

```
import torch
x = torch.tensor([1, 2, 3, 4])
print(f"Original: {x}")
print(f"Cumsum: {x.cumsum(dim=0)}")
```

실행 결과:

```
Original: tensor([1, 2, 3, 4])
Cumsum: tensor([ 1,  3,  6, 10])
```

x.norm(p, dim) **# Lp norm** *Lp* 노름을 계산합니다. *L2*(유클리드) 가 기본값입니다. *L1* 은 맨해튼 거리입니다.

예제:

```
import torch
x = torch.tensor([3.0, 4.0])
print(f"Vector: {x}")
print(f"L2 norm: {x.norm():.4f}")  # sqrt(9+16) = 5
print(f"L1 norm: {x.norm(p=1):.4f}")  # 3+4 = 7
```

실행 결과:

```
Vector: tensor([3., 4.])
L2 norm: 5.0000
L1 norm: 7.0000
```

∫ 수학 함수

`torch.sin(x), cos(x), tan(x)` 삼각 함수입니다. 입력은 라디안입니다.

예제:

```
import torch
import math
x = torch.tensor([0, math.pi/2, math.pi])
print(f"x: {x}")
print(f"sin(x): {torch.sin(x)}")
print(f"cos(x): {torch.cos(x)}")
```

실행 결과:

```
x: tensor([0.0000, 1.5708, 3.1416])
sin(x): tensor([ 0.0000e+00,  1.0000e+00, -8.7423e-08])
cos(x): tensor([ 1.0000e+00, -4.3711e-08, -1.0000e+00])
```

`torch.asin(x), acos(x), atan(x)` 역삼각 함수입니다. 라디안을 반환합니다.

예제:

```
import torch
x = torch.tensor([0.0, 0.5, 1.0])
print(f"x: {x}")
print(f"asin(x): {torch.asin(x)}")
print(f"acos(x): {torch.acos(x)}")
```

실행 결과:

```
x: tensor([0.0000, 0.5000, 1.0000])
asin(x): tensor([0.0000, 0.5236, 1.5708])
acos(x): tensor([1.5708, 1.0472, 0.0000])
```

`torch.sinh(x), cosh(x), tanh(x)` 쌍곡선 함수입니다. *tanh* 는 활성화 함수로 자주 사용됩니다.

예제:

```
import torch
x = torch.tensor([-1.0, 0.0, 1.0])
print(f"x: {x}")
print(f"tanh(x): {torch.tanh(x)}")
```

실행 결과:

```
x: tensor([-1.,  0.,  1.])
tanh(x): tensor([-0.7616,  0.0000,  0.7616])
```

`torch.exp(x), log(x), log10(x)` 지수 및 로그 함수입니다. *log* 는 자연로그 *(밑 e)* 입니다.

예제:

```python
import torch
x = torch.tensor([1.0, 2.0, 3.0])
print(f"x: {x}")
print(f"exp(x): {torch.exp(x)}")
print(f"log(exp(x)): {torch.log(torch.exp(x))}")
```

실행 결과:

```
x: tensor([1., 2., 3.])
exp(x): tensor([ 2.7183,  7.3891, 20.0855])
log(exp(x)): tensor([1.0000, 2.0000, 3.0000])
```

`torch.sqrt(x), rsqrt(x)` 제곱근과 역제곱근 *(1/sqrt(x))* 입니다.

예제:

```python
import torch
x = torch.tensor([1.0, 4.0, 9.0])
print(f"x: {x}")
print(f"sqrt(x): {torch.sqrt(x)}")
print(f"rsqrt(x): {torch.rsqrt(x)}")  # 1/sqrt(x)
```

실행 결과:

```
x: tensor([1., 4., 9.])
sqrt(x): tensor([1., 2., 3.])
rsqrt(x): tensor([1.0000, 0.5000, 0.3333])
```

`torch.floor(x), ceil(x), round(x)` 반올림 함수입니다. *floor* 는 내림, *ceil* 은 올림입니다.

예제:

```python
import torch
x = torch.tensor([1.2, 2.5, 3.7])
print(f"x: {x}")
print(f"floor(x): {torch.floor(x)}")
print(f"ceil(x): {torch.ceil(x)}")
print(f"round(x): {torch.round(x)}")
```

실행 결과:

```
x: tensor([1.2000, 2.5000, 3.7000])
floor(x): tensor([1., 2., 3.])
ceil(x): tensor([2., 3., 4.])
round(x): tensor([1., 2., 4.])
```

`torch.clamp(x, min, max)` 값을 *[min, max]* 범위로 제한합니다. 범위 밖의 값은 경계값으로 설정됩니다.

예제:

```
import torch
x = torch.tensor([-2, 0, 3, 5, 10])
result = torch.clamp(x, min=0, max=5)
print(f"Original: {x}")
print(f"Clamped [0,5]: {result}")
```

실행 결과:

```
Original: tensor([-2,  0,  3,  5, 10])
Clamped [0,5]: tensor([0, 0, 3, 5, 5])
```

`torch.sign(x)` 각 요소의 부호에 따라 *-1, 0,* 또는 *1* 을 반환합니다.

예제:

```
import torch
x = torch.tensor([-3, 0, 5])
print(f"x: {x}")
print(f"sign(x): {torch.sign(x)}")
```

실행 결과:

```
x: tensor([-3,  0,  5])
sign(x): tensor([-1,  0,  1])
```

`torch.sigmoid(x)` 시그모이드 함수*: 1/(1+e^-x).* 값을 *(0, 1)* 로 매핑합니다. 이진 분류에 사용됩니다.

예제:

```
import torch
x = torch.tensor([-2.0, 0.0, 2.0])
print(f"x: {x}")
print(f"sigmoid(x): {torch.sigmoid(x)}")
```

실행 결과:

```
x: tensor([-2.,  0.,  2.])
sigmoid(x): tensor([0.1192, 0.5000, 0.8808])
```

◻ 선형대수

`torch.mm(a, b)  # 2D matrix mult` *2D* 텐서의 행렬 곱셈입니다. 배치나 고차원은 *matmul* 을 사용하세요.

예제:

```
import torch
a = torch.tensor([[1, 2], [3, 4]])
b = torch.tensor([[5, 6], [7, 8]])
result = torch.mm(a, b)
print("A @ B =")
print(result)
```

실행 결과:

```
A @ B =
tensor([[19, 22],
```

```
        [43, 50]])
```

`torch.bmm(a, b)  # batch mm` 배치 행렬 곱셈입니다. 첫 번째 차원이 배치 크기입니다.

예제:

```
import torch
a = torch.randn(10, 3, 4)  # batch of 10 matrices
b = torch.randn(10, 4, 5)
result = torch.bmm(a, b)
print(f"Batch shapes: {a.shape} @ {b.shape} = {result.shape}")
```

실행 결과:

```
Batch shapes: torch.Size([10, 3, 4]) @ torch.Size([10, 4, 5]) = torch.Size([10, 3, 5])
```

`torch.mv(mat, vec)  # matrix-vector` 행렬-벡터 곱셈입니다. *vec* 는 열벡터로 취급됩니다.

예제:

```
import torch
mat = torch.tensor([[1, 2], [3, 4]])
vec = torch.tensor([1, 1])
result = torch.mv(mat, vec)
print(f"Matrix @ vector = {result}")
```

실행 결과:

```
Matrix @ vector = tensor([3, 7])
```

`torch.dot(a, b)  # 1D dot product` *1D* 텐서의 내적입니다. 요소별 곱의 합입니다.

예제:

```
import torch
a = torch.tensor([1.0, 2.0, 3.0])
b = torch.tensor([4.0, 5.0, 6.0])
result = torch.dot(a, b)
print(f"{a} . {b} = {result}")
```

실행 결과:

```
tensor([1., 2., 3.]) . tensor([4., 5., 6.]) = 32.0
```

`torch.det(x)  # determinant` 정사각 행렬의 행렬식을 계산합니다.

예제:

```
import torch
x = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
det = torch.det(x)
print("Matrix:")
print(x)
print(f"Determinant: {det:.4f}")
```

실행 결과:

```
Matrix:
tensor([[1., 2.],
```

```
        [3., 4.]])
Determinant: -2.0000
```

`torch.inverse(x)  # matrix inverse` 행렬의 역행렬을 계산합니다. *A @ A^-1* = 단위행렬입니다.

예제:

```python
import torch
x = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
inv = torch.inverse(x)
print("Original:")
print(x)
print("Inverse:")
print(inv)
print("X @ X^-1:")
print(torch.mm(x, inv))
```

실행 결과:

```
Original:
tensor([[1., 2.],
        [3., 4.]])
Inverse:
tensor([[-2.0000,  1.0000],
        [ 1.5000, -0.5000]])
X @ X^-1:
tensor([[1., 0.],
        [0., 1.]])
```

`torch.svd(x)  # singular value decomp` 특이값 분해입니다. *X = U @ diag(S) @ V^T*

예제:

```python
import torch
x = torch.tensor([[1.0, 2.0], [3.0, 4.0], [5.0, 6.0]])
U, S, V = torch.svd(x)
print(f"Original shape: {x.shape}")
print(f"U: {U.shape}, S: {S.shape}, V: {V.shape}")
print(f"Singular values: {S}")
```

실행 결과:

```
Original shape: torch.Size([3, 2])
U: torch.Size([3, 2]), S: torch.Size([2]), V: torch.Size([2, 2])
Singular values: tensor([9.5255, 0.5143])
```

`torch.eig(x)  # eigenvalues` 고유값을 계산합니다. 일반: *linalg.eig*, 대칭: *linalg.eigvalsh* 를 사용하세요.

예제:

```python
import torch
x = torch.tensor([[1.0, 2.0], [2.0, 1.0]])
eigenvalues = torch.linalg.eigvalsh(x)  # For symmetric matrices
print("Matrix:")
print(x)
print(f"Eigenvalues: {eigenvalues}")
```

실행 결과:

```
Matrix:
tensor([[1., 2.],
        [2., 1.]])
Eigenvalues: tensor([-1.,  3.])
```

`torch.linalg.norm(x, ord)` 행렬 또는 벡터 노름을 계산합니다. 프로베니우스가 행렬의 기본값입니다.

예제:

```
import torch
x = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
print("Matrix:")
print(x)
print(f"Frobenius norm: {torch.linalg.norm(x):.4f}")
print(f"L1 norm: {torch.linalg.norm(x, ord=1):.4f}")
```

실행 결과:

```
Matrix:
tensor([[1., 2.],
        [3., 4.]])
Frobenius norm: 5.4772
L1 norm: 6.0000
```

`torch.linalg.solve(A, b)` 선형 시스템 $Ax = b$ 를 풉니다. 역행렬 계산보다 안정적입니다.

예제:

```
import torch
A = torch.tensor([[3.0, 1.0], [1.0, 2.0]])
b = torch.tensor([9.0, 8.0])
x = torch.linalg.solve(A, b)
print(f"Solving Ax = b")
print(f"x = {x}")
print(f"Verify A@x = {A @ x}")
```

실행 결과:

```
Solving Ax = b
x = tensor([2., 3.])
Verify A@x = tensor([9., 8.])
```

`torch.trace(x)  # sum of diagonal` 대각 요소의 합입니다. 단위행렬의 *trace* 는 차원입니다.

예제:

```
import torch
x = torch.tensor([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print("Matrix:")
print(x)
print(f"Trace: {torch.trace(x)}")  # 1+5+9 = 15
```

실행 결과:

```
Matrix:
```

```
tensor([[1, 2, 3],
        [4, 5, 6],
        [7, 8, 9]])
Trace: 15
```

`torch.outer(a, b)  # outer product` 두 벡터의 외적입니다. 결과 형태는 *(len(a), len(b))* 입니다.

예제:

```python
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([4, 5])
result = torch.outer(a, b)
print(f"a: {a}, b: {b}")
print("Outer product:")
print(result)
```

실행 결과:

```
a: tensor([1, 2, 3]), b: tensor([4, 5])
Outer product:
tensor([[ 4,  5],
        [ 8, 10],
        [12, 15]])
```

▯ 신경망 함수

활성화 함수는 신경망에 비선형성을 추가합니다.

`F.relu(x)  # max(0, x)` *ReLU(Rectified Linear Unit). max(0, x)* 를 반환합니다. 가장 일반적인 활성화 함수입니다.

예제:

```python
import torch
import torch.nn.functional as F
x = torch.tensor([-2.0, -1.0, 0.0, 1.0, 2.0])
print(f"Input: {x}")
print(f"ReLU:  {F.relu(x)}")
```

실행 결과:

```
Input: tensor([-2., -1.,  0.,  1.,  2.])
ReLU:  tensor([0., 0., 0., 1., 2.])
```

`F.leaky_relu(x, neg_slope)` *ReLU* 와 비슷하지만 작은 음수값을 허용합니다. *'dying ReLU'*문제를 방지합니다.

예제:

```python
import torch
import torch.nn.functional as F
x = torch.tensor([-2.0, -1.0, 0.0, 1.0, 2.0])
print(f"Input: {x}")
print(f"Leaky ReLU: {F.leaky_relu(x, 0.1)}")
```

실행 결과:

```
Input: tensor([-2., -1.,  0.,  1.,  2.])
Leaky ReLU: tensor([-0.2000, -0.1000,  0.0000,  1.0000,  2.0000])
```

F.gelu(x) # Gaussian Error 가우시안 오류 선형 단위입니다. 트랜스포머 *(BERT, GPT)* 에서 사용됩니다.

예제:

```
import torch
import torch.nn.functional as F
x = torch.tensor([-2.0, -1.0, 0.0, 1.0, 2.0])
print(f"Input: {x}")
print(f"GELU:  {F.gelu(x)}")
```

실행 결과:

```
Input: tensor([-2., -1.,  0.,  1.,  2.])
GELU:  tensor([-0.0455, -0.1587,  0.0000,  0.8413,  1.9545])
```

F.sigmoid(x) # 1/(1+e^-x) 값을 *(0, 1)* 로 매핑합니다. 이진 분류 출력에 사용됩니다.

예제:

```
import torch
import torch.nn.functional as F
x = torch.tensor([-2.0, 0.0, 2.0])
print(f"Input: {x}")
print(f"Sigmoid: {F.sigmoid(x)}")
```

실행 결과:

```
Input: tensor([-2.,  0.,  2.])
Sigmoid: tensor([0.1192, 0.5000, 0.8808])
```

F.tanh(x) # hyperbolic tan 값을 *(-1, 1)* 로 매핑합니다. *0* 중심이라 은닉층에서 *sigmoid* 보다 좋습니다.

예제:

```
import torch
import torch.nn.functional as F
x = torch.tensor([-2.0, 0.0, 2.0])
print(f"Input: {x}")
print(f"Tanh: {F.tanh(x)}")
```

실행 결과:

```
Input: tensor([-2.,  0.,  2.])
Tanh: tensor([-0.9640,  0.0000,  0.9640])
```

F.softmax(x, dim) # probabilities 로짓을 확률 *(합이 1)* 로 변환합니다. 다중 클래스 출력에 사용됩니다.

예제:

```
import torch
import torch.nn.functional as F
logits = torch.tensor([2.0, 1.0, 0.1])
```

```
probs = F.softmax(logits, dim=0)
print(f"Logits: {logits}")
print(f"Softmax: {probs}")
print(f"Sum: {probs.sum():.4f}")
```

실행 결과:

```
Logits: tensor([2.0000, 1.0000, 0.1000])
Softmax: tensor([0.6590, 0.2424, 0.0986])
Sum: 1.0000
```

`F.log_softmax(x, dim)` *softmax* 의 로그입니다. 수치적으로 더 안정적입니다. *NLLLoss* 와 함께 사용됩니다.

예제:

```
import torch
import torch.nn.functional as F
logits = torch.tensor([2.0, 1.0, 0.1])
log_probs = F.log_softmax(logits, dim=0)
print(f"Logits: {logits}")
print(f"Log softmax: {log_probs}")
```

실행 결과:

```
Logits: tensor([2.0000, 1.0000, 0.1000])
Log softmax: tensor([-0.4170, -1.4170, -2.3170])
```

정규화 기법은 학습 중 과적합을 방지합니다.

`F.dropout(x, p, training)` 확률 *p* 로 요소를 무작위로 *0* 으로 만듭니다. 학습 중에만 활성화됩니다.

예제:

```
import torch
import torch.nn.functional as F
torch.manual_seed(42)
x = torch.ones(10)
dropped = F.dropout(x, p=0.5, training=True)
print(f"Original: {x}")
print(f"Dropout (p=0.5): {dropped}")
```

실행 결과:

```
Original: tensor([1., 1., 1., 1., 1., 1., 1., 1., 1., 1.])
Dropout (p=0.5): tensor([2., 2., 2., 2., 0., 2., 0., 0., 2., 2.])
```

`F.batch_norm(x, ...)  # normalize` 배치 차원에서 정규화합니다. 학습을 안정화시킵니다.

예제:

```
import torch
import torch.nn.functional as F
x = torch.randn(2, 3, 4, 4)  # batch, channels, H, W
mean = torch.zeros(3)
var = torch.ones(3)
result = F.batch_norm(x, mean, var)
```

```
print(f"Input shape: {x.shape}")
print(f"After batch_norm: mean~{result.mean():.4f}, std~{result.std():.4f}")
```

실행 결과:

```
Input shape: torch.Size([2, 3, 4, 4])
After batch_norm: mean~0.1266, std~0.9259
```

F.layer_norm(x, shape) 지정된 차원에서 정규화합니다. 트랜스포머에서 사용됩니다.

예제:

```
import torch
import torch.nn.functional as F
x = torch.randn(2, 3, 4)
result = F.layer_norm(x, [3, 4])
print(f"Input shape: {x.shape}")
print(f"After layer_norm: mean~{result.mean():.4f}, std~{result.std():.4f}")
```

실행 결과:

```
Input shape: torch.Size([2, 3, 4])
After layer_norm: mean~0.0000, std~1.0215
```

× 손실 함수

F.mse_loss(pred, target) 평균 제곱 오차입니다. 차이의 제곱의 평균입니다. 회귀에 사용됩니다.

예제:

```
import torch
import torch.nn.functional as F
pred = torch.tensor([1.0, 2.0, 3.0])
target = torch.tensor([1.5, 2.0, 2.5])
loss = F.mse_loss(pred, target)
print(f"Prediction: {pred}")
print(f"Target: {target}")
print(f"MSE Loss: {loss:.4f}")
```

실행 결과:

```
Prediction: tensor([1., 2., 3.])
Target: tensor([1.5000, 2.0000, 2.5000])
MSE Loss: 0.1667
```

F.l1_loss(pred, target) 평균 절대 오차입니다. *MSE* 보다 이상치에 덜 민감합니다.

예제:

```
import torch
import torch.nn.functional as F
pred = torch.tensor([1.0, 2.0, 3.0])
target = torch.tensor([1.5, 2.0, 2.5])
loss = F.l1_loss(pred, target)
print(f"Prediction: {pred}")
```

```
print(f"Target: {target}")
print(f"L1 Loss: {loss:.4f}")
```

실행 결과:

```
Prediction: tensor([1., 2., 3.])
Target: tensor([1.5000, 2.0000, 2.5000])
L1 Loss: 0.3333
```

F.cross_entropy(logits, labels) *log_softmax* 와 *NLLLoss* 를 결합합니다. 다중 클래스 분류의 표준입니다.

예제:

```
import torch
import torch.nn.functional as F
logits = torch.tensor([[2.0, 0.5, 0.1], [0.1, 2.0, 0.5]])
labels = torch.tensor([0, 1])  # Class indices
loss = F.cross_entropy(logits, labels)
print(f"Logits: {logits}")
print(f"Labels: {labels}")
print(f"Cross Entropy Loss: {loss:.4f}")
```

실행 결과:

```
Logits: tensor([[2.0000, 0.5000, 0.1000],
        [0.1000, 2.0000, 0.5000]])
Labels: tensor([0, 1])
Cross Entropy Loss: 0.3168
```

F.nll_loss(log_probs, labels) 음의 로그 우도입니다. *log_softmax* 출력과 함께 사용합니다.

예제:

```
import torch
import torch.nn.functional as F
log_probs = F.log_softmax(torch.tensor([[2.0, 0.5], [0.5, 2.0]]), dim=1)
labels = torch.tensor([0, 1])
loss = F.nll_loss(log_probs, labels)
print(f"NLL Loss: {loss:.4f}")
```

실행 결과:

```
NLL Loss: 0.2014
```

F.binary_cross_entropy(pred, target) 이진 교차 엔트로피입니다. *sigmoid* 출력의 이진 분류에 사용됩니다.

예제:

```
import torch
import torch.nn.functional as F
pred = torch.tensor([0.8, 0.4, 0.9])  # Probabilities
target = torch.tensor([1.0, 0.0, 1.0])  # Binary labels
loss = F.binary_cross_entropy(pred, target)
print(f"Pred: {pred}")
```

```
print(f"Target: {target}")
print(f"BCE Loss: {loss:.4f}")
```

실행 결과:

```
Pred: tensor([0.8000, 0.4000, 0.9000])
Target: tensor([1., 0., 1.])
BCE Loss: 0.2798
```

F.kl_div(log_pred, target) 쿨백-라이블러 발산입니다. 분포 간의 차이를 측정합니다.

예제:

```
import torch
import torch.nn.functional as F
log_pred = F.log_softmax(torch.tensor([0.5, 0.3, 0.2]), dim=0)
target = F.softmax(torch.tensor([0.4, 0.4, 0.2]), dim=0)
loss = F.kl_div(log_pred, target, reduction='sum')
print(f"KL Divergence: {loss:.4f}")
```

실행 결과:

```
KL Divergence: 0.0035
```

F.cosine_similarity(x1, x2, dim) 벡터 간의 각도를 측정합니다. *1* = 같은 방향, *-1* = 반대 방향.

예제:

```
import torch
import torch.nn.functional as F
x1 = torch.tensor([[1.0, 0.0, 0.0]])
x2 = torch.tensor([[1.0, 1.0, 0.0]])
sim = F.cosine_similarity(x1, x2)
print(f"x1: {x1}")
print(f"x2: {x2}")
print(f"Cosine similarity: {sim}")
```

실행 결과:

```
x1: tensor([[1., 0., 0.]])
x2: tensor([[1., 1., 0.]])
Cosine similarity: tensor([0.7071])
```

F.triplet_margin_loss(...) 유사한 항목은 가깝게, 다른 항목은 멀게 임베딩을 학습합니다.

예제:

```
import torch
import torch.nn.functional as F
anchor = torch.randn(3, 128)
positive = anchor + 0.1 * torch.randn(3, 128)
negative = torch.randn(3, 128)
loss = F.triplet_margin_loss(anchor, positive, negative, margin=1.0)
print(f"Triplet loss: {loss:.4f}")
```

실행 결과:

```
Triplet loss: 0.0000
```

▯ 풀링과 컨볼루션

F.max_pool2d(x, kernel_size) 각 윈도우에서 최대값을 취하여 다운샘플링합니다. 공간 차원을 줄입니다.

예제:

```
import torch
import torch.nn.functional as F
x = torch.arange(16).float().view(1, 1, 4, 4)
print("Input (4x4):")
print(x.squeeze())
result = F.max_pool2d(x, kernel_size=2)
print("MaxPool2d (kernel=2):")
print(result.squeeze())
```

실행 결과:

```
Input (4x4):
tensor([[ 0.,  1.,  2.,  3.],
        [ 4.,  5.,  6.,  7.],
        [ 8.,  9., 10., 11.],
        [12., 13., 14., 15.]])
MaxPool2d (kernel=2):
tensor([[ 5.,  7.],
        [13., 15.]])
```

F.avg_pool2d(x, kernel_size) 각 윈도우를 평균내어 다운샘플링합니다. *max pooling* 보다 부드럽습니다.

예제:

```
import torch
import torch.nn.functional as F
x = torch.arange(16).float().view(1, 1, 4, 4)
print("Input (4x4):")
print(x.squeeze())
result = F.avg_pool2d(x, kernel_size=2)
print("AvgPool2d (kernel=2):")
print(result.squeeze())
```

실행 결과:

```
Input (4x4):
tensor([[ 0.,  1.,  2.,  3.],
        [ 4.,  5.,  6.,  7.],
        [ 8.,  9., 10., 11.],
        [12., 13., 14., 15.]])
AvgPool2d (kernel=2):
tensor([[ 2.5000,  4.5000],
        [10.5000, 12.5000]])
```

F.adaptive_max_pool2d(x, output_size) 입력 크기와 관계없이 정확한 출력 크기로 풀링합니다.

예제:

```
import torch
import torch.nn.functional as F
x = torch.randn(1, 1, 7, 7)
result = F.adaptive_max_pool2d(x, output_size=(2, 2))
print(f"Input: {x.shape} -> Output: {result.shape}")
```

실행 결과:

```
Input: torch.Size([1, 1, 7, 7]) -> Output: torch.Size([1, 1, 2, 2])
```

`F.conv2d(x, weight, bias)` *2D* 컨볼루션입니다. *CNN* 의 핵심 연산입니다. 공간 특징을 추출합니다.

예제:

```
import torch
import torch.nn.functional as F
x = torch.randn(1, 1, 5, 5)  # batch, in_ch, H, W
weight = torch.randn(1, 1, 3, 3)  # out_ch, in_ch, kH, kW
result = F.conv2d(x, weight)
print(f"Input: {x.shape}")
print(f"Weight: {weight.shape}")
print(f"Output: {result.shape}")
```

실행 결과:

```
Input: torch.Size([1, 1, 5, 5])
Weight: torch.Size([1, 1, 3, 3])
Output: torch.Size([1, 1, 3, 3])
```

`F.conv_transpose2d(x, weight)` 전치 컨볼루션 *(*디컨볼루션*)* 입니다. 공간 차원을 업샘플링합니다.

예제:

```
import torch
import torch.nn.functional as F
x = torch.randn(1, 1, 3, 3)
weight = torch.randn(1, 1, 3, 3)
result = F.conv_transpose2d(x, weight)
print(f"Input: {x.shape}")
print(f"Output: {result.shape}")
```

실행 결과:

```
Input: torch.Size([1, 1, 3, 3])
Output: torch.Size([1, 1, 5, 5])
```

`F.interpolate(x, size, mode)` 보간을 사용하여 텐서 크기를 조정합니다. 모드*: nearest, bilinear, bicubic.*

예제:

```
import torch
import torch.nn.functional as F
x = torch.randn(1, 1, 4, 4)
result = F.interpolate(x, size=(8, 8), mode='bilinear', align_corners=False)
print(f"Input: {x.shape} -> Output: {result.shape}")
```

실행 결과:

```
Input: torch.Size([1, 1, 4, 4]) -> Output: torch.Size([1, 1, 8, 8])
```

F.pad(x, pad, mode) 텐서 주위에 패딩을 추가합니다. 컨볼루션에 사용됩니다. 모드*: constant, reflect, replicate.*

예제:

```
import torch
import torch.nn.functional as F
x = torch.arange(4).float().view(1, 1, 2, 2)
print("Input:")
print(x.squeeze())
padded = F.pad(x, (1, 1, 1, 1), mode='constant', value=0)
print("Padded (1 on each side):")
print(padded.squeeze())
```

실행 결과:

```
Input:
tensor([[0., 1.],
        [2., 3.]])
Padded (1 on each side):
tensor([[0., 0., 0., 0.],
        [0., 0., 1., 0.],
        [0., 2., 3., 0.],
        [0., 0., 0., 0.]])
```

∞ 고급 연산

torch.einsum('ij,jk->ik', a, b) 아인슈타인 합산입니다. 많은 텐서 연산을 위한 유연한 표기법입니다.

예제:

```
import torch
a = torch.randn(2, 3)
b = torch.randn(3, 4)
result = torch.einsum('ij,jk->ik', a, b)
print(f"einsum('ij,jk->ik'): {a.shape} x {b.shape} = {result.shape}")
# Verify it's matrix multiplication
print(f"Same as matmul: {torch.allclose(result, a @ b)}")
```

실행 결과:

```
einsum('ij,jk->ik'): torch.Size([2, 3]) x torch.Size([3, 4]) = torch.Size([2, 4])
Same as matmul: True
```

torch.topk(x, k, dim) # top k values *k* 개의 가장 큰 값과 인덱스를 반환합니다. 전체 정렬보다 빠릅니다.

예제:

```
import torch
x = torch.tensor([1, 5, 3, 9, 2, 7])
vals, idxs = torch.topk(x, k=3)
print(f"Input: {x}")
```

```
print(f"Top 3 values: {vals}")
print(f"Top 3 indices: {idxs}")
```

실행 결과:

```
Input: tensor([1, 5, 3, 9, 2, 7])
Top 3 values: tensor([9, 7, 5])
Top 3 indices: tensor([3, 5, 1])
```

`torch.sort(x, dim)  # sorted values` 차원을 따라 텐서를 정렬합니다. 값과 원래 인덱스를 반환합니다.

예제:

```
import torch
x = torch.tensor([3, 1, 4, 1, 5, 9])
vals, idxs = torch.sort(x)
print(f"Original: {x}")
print(f"Sorted: {vals}")
print(f"Indices: {idxs}")
```

실행 결과:

```
Original: tensor([3, 1, 4, 1, 5, 9])
Sorted: tensor([1, 1, 3, 4, 5, 9])
Indices: tensor([1, 3, 0, 2, 4, 5])
```

`torch.argsort(x, dim)  # sort indices` 텐서를 정렬할 인덱스를 반환합니다.

예제:

```
import torch
x = torch.tensor([3, 1, 4, 1, 5])
idxs = torch.argsort(x)
print(f"Original: {x}")
print(f"Argsort: {idxs}")
print(f"Sorted via indices: {x[idxs]}")
```

실행 결과:

```
Original: tensor([3, 1, 4, 1, 5])
Argsort: tensor([1, 3, 0, 2, 4])
Sorted via indices: tensor([1, 1, 3, 4, 5])
```

`torch.unique(x)  # unique values` 고유한 요소를 반환합니다. 빈도를 위한 선택적 *return_counts*가 있습니다.

예제:

```
import torch
x = torch.tensor([1, 2, 2, 3, 1, 3, 3, 4])
unique = torch.unique(x)
print(f"Original: {x}")
print(f"Unique: {unique}")
```

실행 결과:

```
Original: tensor([1, 2, 2, 3, 1, 3, 3, 4])
Unique: tensor([1, 2, 3, 4])
```

`torch.cat([t1, t2], dim)` **`# concat`** 기존 차원을 따라 텐서를 연결합니다. *dim* 을 제외한 형태가 일치해야 합니다.

예제:

```
import torch
a = torch.tensor([[1, 2], [3, 4]])
b = torch.tensor([[5, 6]])
result = torch.cat([a, b], dim=0)
print("Concatenated along dim 0:")
print(result)
```

실행 결과:

```
Concatenated along dim 0:
tensor([[1, 2],
        [3, 4],
        [5, 6]])
```

`torch.stack([t1, t2], dim)` **`# new dim`** 새로운 차원을 따라 텐서를 쌓습니다. 모든 텐서가 같은 형태여야 합니다.

예제:

```
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([4, 5, 6])
result = torch.stack([a, b], dim=0)
print("Stacked (creates new dim):")
print(result)
print(f"Shape: {result.shape}")
```

실행 결과:

```
Stacked (creates new dim):
tensor([[1, 2, 3],
        [4, 5, 6]])
Shape: torch.Size([2, 3])
```

`torch.split(x, size, dim)` **`# split`** 텐서를 지정된 크기의 청크로 분할합니다. 마지막 청크는 더 작을 수 있습니다.

예제:

```
import torch
x = torch.arange(10)
splits = torch.split(x, 3)
print(f"Original: {x}")
print("Split into chunks of 3:")
for i, s in enumerate(splits):
    print(f"  {i}: {s}")
```

실행 결과:

```
Original: tensor([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
Split into chunks of 3:
  0: tensor([0, 1, 2])
  1: tensor([3, 4, 5])
  2: tensor([6, 7, 8])
  3: tensor([9])
```

`torch.chunk(x, chunks, dim)` **`# chunks`** 텐서를 지정된 수의 청크로 분할합니다.

예제:

```
import torch
x = torch.arange(12)
chunks = torch.chunk(x, 4)
print(f"Original: {x}")
print("Split into 4 chunks:")
for i, c in enumerate(chunks):
    print(f"  {i}: {c}")
```

실행 결과:

```
Original: tensor([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11])
Split into 4 chunks:
  0: tensor([0, 1, 2])
  1: tensor([3, 4, 5])
  2: tensor([6, 7, 8])
  3: tensor([ 9, 10, 11])
```

`torch.broadcast_to(x, shape)` 텐서를 명시적으로 새로운 형태로 브로드캐스트합니다. 데이터를 복사하지 않습니다.

예제:

```
import torch
x = torch.tensor([1, 2, 3])
result = torch.broadcast_to(x, (3, 3))
print(f"Original: {x}")
print("Broadcast to (3, 3):")
print(result)
```

실행 결과:

```
Original: tensor([1, 2, 3])
Broadcast to (3, 3):
tensor([[1, 2, 3],
        [1, 2, 3],
        [1, 2, 3]])
```

`torch.flatten(x, start, end)` 텐서를 평탄화합니다. 부분 평탄화를 위한 선택적 시작/끝 차원이 있습니다.

예제:

```
import torch
x = torch.randn(2, 3, 4)
flat = torch.flatten(x)
partial = torch.flatten(x, start_dim=1)
```

```
print(f"Original: {x.shape}")
print(f"Fully flat: {flat.shape}")
print(f"Flatten from dim 1: {partial.shape}")
```

실행 결과:

```
Original: torch.Size([2, 3, 4])
Fully flat: torch.Size([24])
Flatten from dim 1: torch.Size([2, 12])
```

∂ 자동미분

`x.requires_grad_(True)` 텐서의 그래디언트 추적을 활성화합니다. 역전파에 필요합니다.

예제:

```
import torch
x = torch.tensor([1.0, 2.0, 3.0])
print(f"Before: requires_grad = {x.requires_grad}")
x.requires_grad_(True)
print(f"After: requires_grad = {x.requires_grad}")
```

실행 결과:

```
Before: requires_grad = False
After: requires_grad = True
```

`y.backward()` 역전파를 통해 그래디언트를 계산합니다. *.grad* 속성에 저장됩니다.

예제:

```
import torch
x = torch.tensor([2.0, 3.0], requires_grad=True)
y = (x ** 2).sum()  # y = x1^2 + x2^2
y.backward()
print(f"x = {x}")
print(f"y = x^2.sum() = {y}")
print(f"dy/dx = {x.grad}")  # 2*x
```

실행 결과:

```
x = tensor([2., 3.], requires_grad=True)
y = x^2.sum() = 13.0
dy/dx = tensor([4., 6.])
```

`x.grad  # gradient` *backward()* 후 누적된 그래디언트를 저장합니다. *zero_grad()* 로 리셋합니다.

예제:

```
import torch
x = torch.tensor(3.0, requires_grad=True)
y = x ** 2
y.backward()
print(f"x = {x}")
print(f"y = x^2 = {y}")
print(f"dy/dx = {x.grad}")
```

실행 결과:

```
x = 3.0
y = x^2 = 9.0
dy/dx = 6.0
```

x.detach() 계산 그래프에서 분리된 텐서를 반환합니다. 그래디언트 흐름을 중단합니다.

예제:

```
import torch
x = torch.tensor([1.0, 2.0], requires_grad=True)
y = x * 2
z = y.detach()
print(f"y requires_grad: {y.requires_grad}")
print(f"z requires_grad: {z.requires_grad}")
```

실행 결과:

```
y requires_grad: True
z requires_grad: False
```

x.clone() 같은 데이터의 복사본을 생성합니다. 수정해도 원본에 영향을 주지 않습니다.

예제:

```
import torch
x = torch.tensor([1.0, 2.0])
y = x.clone()
y[0] = 99
print(f"Original x: {x}")
print(f"Cloned y: {y}")
```

실행 결과:

```
Original x: tensor([1., 2.])
Cloned y: tensor([99.,  2.])
```

with torch.no_grad(): 그래디언트 계산을 비활성화하는 컨텍스트 관리자입니다. 추론용입니다.

예제:

```
import torch
x = torch.tensor([1.0, 2.0], requires_grad=True)
with torch.no_grad():
    y = x * 2
    print(f"Inside no_grad: y.requires_grad = {y.requires_grad}")
z = x * 2
print(f"Outside: z.requires_grad = {z.requires_grad}")
```

실행 결과:

```
Inside no_grad: y.requires_grad = False
Outside: z.requires_grad = True
```

torch.autograd.grad(y, x) *.grad* 를 수정하지 않고 그래디언트를 계산합니다. *2* 차 미분에 유용합니다.

예제:

```
import torch
x = torch.tensor([1.0, 2.0], requires_grad=True)
y = (x ** 3).sum()  # y = x1^3 + x2^3
grad = torch.autograd.grad(y, x)
print(f"x = {x}")
print(f"dy/dx = {grad[0]}")  # 3*x^2
```

실행 결과:

```
x = tensor([1., 2.], requires_grad=True)
dy/dx = tensor([ 3., 12.])
```

`optimizer.zero_grad()` 그래디언트를 *0* 으로 리셋합니다. 각 *backward pass* 전에 호출합니다.

예제:

```
import torch
import torch.optim as optim
x = torch.tensor([1.0], requires_grad=True)
optimizer = optim.SGD([x], lr=0.1)
y = x ** 2
y.backward()
print(f"Grad before zero_grad: {x.grad}")
optimizer.zero_grad()
print(f"Grad after zero_grad: {x.grad}")
```

실행 결과:

```
Grad before zero_grad: tensor([2.])
Grad after zero_grad: None
```

`optimizer.step()` 계산된 그래디언트를 사용하여 파라미터를 업데이트합니다. *x = x - lr* grad.*

예제:

```
import torch
import torch.optim as optim
x = torch.tensor([5.0], requires_grad=True)
optimizer = optim.SGD([x], lr=0.1)
print(f"Before: x = {x.item():.4f}")
y = x ** 2
y.backward()
optimizer.step()
print(f"After step: x = {x.item():.4f}")
```

실행 결과:

```
Before: x = 5.0000
After step: x = 4.0000
```

`torch.nn.utils.clip_grad_norm_(params, max)` 그래디언트 폭발을 방지하기 위해 그래디언트 노름을 클리핑합니다. *RNN* 에 필수입니다.

예제:

```
import torch
import torch.nn as nn
```

```
model = nn.Linear(10, 5)
x = torch.randn(1, 10)
loss = model(x).sum()
loss.backward()
norm_before = sum(p.grad.norm()**2 for p in model.parameters())**0.5
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
norm_after = sum(p.grad.norm()**2 for p in model.parameters())**0.5
print(f"Grad norm before: {norm_before:.4f}")
print(f"Grad norm after clip: {norm_after:.4f}")
```

실행 결과:

```
Grad norm before: 6.8840
Grad norm after clip: 1.0000
```

디바이스 연산

`torch.cuda.is_available()` *CUDA GPU* 사용 가능 여부를 확인합니다. 디바이스 선택 로직에 사용됩니다.

예제:

```
import torch
print(f"CUDA available: {torch.cuda.is_available()}")
print(f"MPS available: {torch.backends.mps.is_available()}")
```

실행 결과:

```
CUDA available: False
MPS available: True
```

`torch.device('cuda'/'cpu'/'mps')` 디바이스 객체를 생성합니다. *.to(device)* 와 함께 디바이스 배치에 사용됩니다.

예제:

```
import torch
cpu = torch.device('cpu')
print(f"CPU device: {cpu}")
if torch.cuda.is_available():
    gpu = torch.device('cuda')
    print(f"GPU device: {gpu}")
elif torch.backends.mps.is_available():
    mps = torch.device('mps')
    print(f"MPS device: {mps}")
```

실행 결과:

```
CPU device: cpu
MPS device: mps
```

`x.to(device)  # move to device` 텐서를 지정된 디바이스로 이동합니다. *GPU* 학습에 필수입니다.

예제:

```
import torch
x = torch.tensor([1.0, 2.0])
device = torch.device('cpu')
x_dev = x.to(device)
print(f"Tensor on: {x_dev.device}")
```

실행 결과:

```
Tensor on: cpu
```

`x.to(dtype)  # change dtype` 텐서를 다른 데이터 타입으로 변환합니다.

예제:

```
import torch
x = torch.tensor([1, 2, 3])
print(f"Original dtype: {x.dtype}")
x_float = x.to(torch.float32)
print(f"After to(float32): {x_float.dtype}")
```

실행 결과:

```
Original dtype: torch.int64
After to(float32): torch.float32
```

`x.cuda(), x.cpu()  # shortcuts` *CPU* 와 *CUDA* 간 이동을 위한 단축 메서드입니다.

예제:

```
import torch
x = torch.tensor([1.0, 2.0])
print(f"Original device: {x.device}")
x_cpu = x.cpu()
print(f"After cpu(): {x_cpu.device}")
```

실행 결과:

```
Original device: cpu
After cpu(): cpu
```

`x.device  # check current device` 텐서가 저장된 디바이스를 반환합니다.

예제:

```
import torch
x = torch.tensor([1.0, 2.0])
print(f"Device: {x.device}")
print(f"Device type: {x.device.type}")
```

실행 결과:

```
Device: cpu
Device type: cpu
```

`x.is_cuda  # boolean check` 텐서가 *CUDA* 디바이스에 있으면 *True* 를 반환합니다.

예제:

```python
import torch
x = torch.tensor([1.0, 2.0])
print(f"is_cuda: {x.is_cuda}")
```

실행 결과:

```
is_cuda: False
```

torch.cuda.empty_cache() 사용되지 않는 캐시된 *GPU* 메모리를 해제합니다. *OOM* 오류에 도움이 됩니다.

예제:

```python
import torch
# Free unused cached memory (only affects CUDA)
if torch.cuda.is_available():
    torch.cuda.empty_cache()
    print("CUDA cache cleared")
else:
    print("CUDA not available - cache clearing skipped")
```

실행 결과:

```
CUDA not available - cache clearing skipped
```

torch.cuda.device_count() 사용 가능한 *CUDA* 디바이스 수를 반환합니다.

예제:

```python
import torch
if torch.cuda.is_available():
    count = torch.cuda.device_count()
    print(f"Number of GPUs: {count}")
else:
    print("CUDA not available")
```

실행 결과:

```
CUDA not available
```

※ 유틸리티

x.dtype, x.shape, x.size() 기본 텐서 속성입니다. *shape* 와 *size()* 는 동일합니다.

예제:

```python
import torch
x = torch.randn(2, 3, 4)
print(f"dtype: {x.dtype}")
print(f"shape: {x.shape}")
print(f"size(): {x.size()}")
```

실행 결과:

```
dtype: torch.float32
shape: torch.Size([2, 3, 4])
size(): torch.Size([2, 3, 4])
```

`x.numel()  # number of elements` 총 요소 수를 반환합니다. 모든 차원의 곱입니다.

예제:

```
import torch
x = torch.randn(2, 3, 4)
print(f"Shape: {x.shape}")
print(f"Number of elements: {x.numel()}")
```

실행 결과:

```
Shape: torch.Size([2, 3, 4])
Number of elements: 24
```

`x.dim()  # number of dimensions` 텐서의 차원 수 *(랭크)* 를 반환합니다.

예제:

```
import torch
x = torch.randn(2, 3, 4)
print(f"Shape: {x.shape}")
print(f"Dimensions: {x.dim()}")
```

실행 결과:

```
Shape: torch.Size([2, 3, 4])
Dimensions: 3
```

`x.ndimension()  # same as dim()` *dim()* 의 별칭입니다. 차원 수를 반환합니다.

예제:

```
import torch
x = torch.randn(2, 3, 4)
print(f"ndimension(): {x.ndimension()}")
print(f"Same as dim(): {x.dim()}")
```

실행 결과:

```
ndimension(): 3
Same as dim(): 3
```

`x.is_contiguous()` 텐서가 메모리에서 연속적인지 확인합니다. *view()* 에 필요합니다.

예제:

```
import torch
x = torch.randn(3, 4)
y = x.transpose(0, 1)
print(f"Original is_contiguous: {x.is_contiguous()}")
print(f"Transposed is_contiguous: {y.is_contiguous()}")
```

실행 결과:

```
Original is_contiguous: True
Transposed is_contiguous: False
```

`x.float(), x.int(), x.long()` *dtype* 변환을 위한 단축 메서드입니다.

예제:

```
import torch
x = torch.tensor([1, 2, 3])
print(f"Original: {x.dtype}")
print(f"float(): {x.float().dtype}")
print(f"long(): {x.long().dtype}")
```

실행 결과:

```
Original: torch.int64
float(): torch.float32
long(): torch.int64
```

`x.half(), x.double() # fp16, fp64` 반정밀도 *(16* 비트*)* 는 빠른 학습에*, double* 은 높은 정밀도에 사용됩니다*.*

예제:

```
import torch
x = torch.tensor([1.0, 2.0])
print(f"Original: {x.dtype}")
print(f"half() (fp16): {x.half().dtype}")
print(f"double() (fp64): {x.double().dtype}")
```

실행 결과:

```
Original: torch.float32
half() (fp16): torch.float16
double() (fp64): torch.float64
```

`torch.from_numpy(arr)` *NumPy* 배열을 텐서로 변환합니다*.* 원본과 메모리를 공유합니다*.*

예제:

```
import torch
import numpy as np
arr = np.array([1, 2, 3])
x = torch.from_numpy(arr)
print(f"NumPy: {arr}, dtype={arr.dtype}")
print(f"Tensor: {x}, dtype={x.dtype}")
```

실행 결과:

```
NumPy: [1 2 3], dtype=int64
Tensor: tensor([1, 2, 3]), dtype=torch.int64
```

`x.numpy() # CPU only` 텐서를 *NumPy* 배열로 변환합니다*. CPU* 에 있어야 합니다*.*

예제:

```
import torch
x = torch.tensor([1.0, 2.0, 3.0])
arr = x.numpy()
print(f"Tensor: {x}")
print(f"NumPy: {arr}, dtype={arr.dtype}")
```

실행 결과:

```
Tensor: tensor([1., 2., 3.])
NumPy: [1. 2. 3.], dtype=float32
```

`torch.save(obj, path)` 텐서나 모델을 파일에 저장합니다. *pickle* 직렬화를 사용합니다.

예제:

```
import torch
import os
x = torch.tensor([1, 2, 3])
torch.save(x, '/tmp/tensor.pt')
print(f"Saved tensor to /tmp/tensor.pt")
print(f"File size: {os.path.getsize('/tmp/tensor.pt')} bytes")
```

실행 결과:

```
Saved tensor to /tmp/tensor.pt
File size: 1570 bytes
```

`torch.load(path)` 저장된 텐서나 모델을 파일에서 로드합니다.

예제:

```
import torch
torch.save(torch.tensor([1, 2, 3]), '/tmp/tensor.pt')
loaded = torch.load('/tmp/tensor.pt', weights_only=True)
print(f"Loaded: {loaded}")
```

실행 결과:

```
Loaded: tensor([1, 2, 3])
```

`torch.manual_seed(seed)` 재현성을 위해 난수 시드를 설정합니다.

예제:

```
import torch
torch.manual_seed(42)
a = torch.rand(3)
torch.manual_seed(42)
b = torch.rand(3)
print(f"First: {a}")
print(f"Second (same seed): {b}")
print(f"Equal: {torch.equal(a, b)}")
```

실행 결과:

```
First: tensor([0.8823, 0.9150, 0.3829])
Second (same seed): tensor([0.8823, 0.9150, 0.3829])
Equal: True
```

≈ 비교 연산

`torch.eq(a, b) or a == b` 요소별 동등 비교입니다. 불리언 텐서를 반환합니다.

예제:

```
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([1, 0, 3])
print(f"a: {a}")
print(f"b: {b}")
print(f"a == b: {a == b}")
```

실행 결과:

```
a: tensor([1, 2, 3])
b: tensor([1, 0, 3])
a == b: tensor([ True, False,  True])
```

torch.ne(a, b) or a != b 요소별 부등 비교입니다.

예제:

```
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([1, 0, 3])
print(f"a != b: {a != b}")
```

실행 결과:

```
a != b: tensor([False,  True, False])
```

torch.gt(a, b) or a > b 요소별 초과 비교입니다.

예제:

```
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([2, 2, 2])
print(f"a: {a}, b: {b}")
print(f"a > b: {a > b}")
```

실행 결과:

```
a: tensor([1, 2, 3]), b: tensor([2, 2, 2])
a > b: tensor([False, False,  True])
```

torch.lt(a, b) or a < b 요소별 미만 비교입니다.

예제:

```
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([2, 2, 2])
print(f"a < b: {a < b}")
```

실행 결과:

```
a < b: tensor([ True, False, False])
```

torch.ge(a, b) or a >= b 요소별 이상 비교입니다.

예제:

```
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([2, 2, 2])
print(f"a >= b: {a >= b}")
```

실행 결과:

```
a >= b: tensor([False,  True,  True])
```

`torch.le(a, b) or a <= b` 요소별 이하 비교입니다.

예제:

```
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([2, 2, 2])
print(f"a <= b: {a <= b}")
```

실행 결과:

```
a <= b: tensor([ True,  True, False])
```

`torch.allclose(a, b, rtol, atol)` 모든 요소가 허용 오차 내에서 가까운지 확인합니다. *float* 비교용입니다.

예제:

```
import torch
a = torch.tensor([1.0, 2.0])
b = torch.tensor([1.0001, 2.0001])
print(f"a: {a}")
print(f"b: {b}")
print(f"allclose (default tol): {torch.allclose(a, b)}")
print(f"allclose (rtol=1e-3): {torch.allclose(a, b, rtol=1e-3)}")
```

실행 결과:

```
a: tensor([1., 2.])
b: tensor([1.0001, 2.0001])
allclose (default tol): False
allclose (rtol=1e-3): True
```

`torch.isnan(x)` 요소가 *NaN(*숫자가 아님*)* 인 곳에서 *True* 를 반환합니다.

예제:

```
import torch
x = torch.tensor([1.0, float('nan'), 3.0])
print(f"x: {x}")
print(f"isnan: {torch.isnan(x)}")
```

실행 결과:

```
x: tensor([1., nan, 3.])
isnan: tensor([False,  True, False])
```

`torch.isinf(x)` 요소가 무한대인 곳에서 *True* 를 반환합니다.

예제:

```
import torch
x = torch.tensor([1.0, float('inf'), float('-inf')])
print(f"x: {x}")
print(f"isinf: {torch.isinf(x)}")
```

실행 결과:

```
x: tensor([1., inf, -inf])
isinf: tensor([False,  True,  True])
```

`torch.isfinite(x)` 요소가 유한 *(inf* 아님*, nan* 아님*)* 인 곳에서 *True* 를 반환합니다.

예제:

```
import torch
x = torch.tensor([1.0, float('inf'), float('nan')])
print(f"x: {x}")
print(f"isfinite: {torch.isfinite(x)}")
```

실행 결과:

```
x: tensor([1., inf, nan])
isfinite: tensor([ True, False, False])
```

● 텐서 메서드

`x.T # transpose (2D)` *2D* 전치의 약칭입니다. 고차원은 *permute* 를 사용하세요.

예제:

```
import torch
x = torch.tensor([[1, 2, 3], [4, 5, 6]])
print("Original:")
print(x)
print("x.T:")
print(x.T)
```

실행 결과:

```
Original:
tensor([[1, 2, 3],
        [4, 5, 6]])
x.T:
tensor([[1, 4],
        [2, 5],
        [3, 6]])
```

`x.H # conjugate transpose` 복소수 텐서의 에르미트 전치입니다. 전치 + 켤레복소수입니다.

예제:

```
import torch
x = torch.tensor([[1+2j, 3+4j], [5+6j, 7+8j]])
print("Original:")
```

```
print(x)
print("x.H (conjugate transpose):")
print(x.H)
```

실행 결과:

```
Original:
tensor([[1.+2.j, 3.+4.j],
        [5.+6.j, 7.+8.j]])
x.H (conjugate transpose):
tensor([[1.-2.j, 5.-6.j],
        [3.-4.j, 7.-8.j]])
```

`x.real, x.imag  # complex parts` 복소수 텐서의 실수부와 허수부에 접근합니다.

예제:

```
import torch
x = torch.tensor([1+2j, 3+4j])
print(f"Complex: {x}")
print(f"Real: {x.real}")
print(f"Imag: {x.imag}")
```

실행 결과:

```
Complex: tensor([1.+2.j, 3.+4.j])
Real: tensor([1., 3.])
Imag: tensor([2., 4.])
```

`x.abs(), x.neg()  # absolute, negate` 절대값과 부정을 위한 메서드입니다.

예제:

```
import torch
x = torch.tensor([-3, -1, 0, 2, 4])
print(f"x: {x}")
print(f"abs: {x.abs()}")
print(f"neg: {x.neg()}")
```

실행 결과:

```
x: tensor([-3, -1,  0,  2,  4])
abs: tensor([3, 1, 0, 2, 4])
neg: tensor([ 3,  1,  0, -2, -4])
```

`x.reciprocal(), x.pow(n)` 역수 *(1/x)* 와 거듭제곱 연산을 메서드로 제공합니다.

예제:

```
import torch
x = torch.tensor([1.0, 2.0, 4.0])
print(f"x: {x}")
print(f"reciprocal: {x.reciprocal()}")
print(f"pow(2): {x.pow(2)}")
```

실행 결과:

```
x: tensor([1., 2., 4.])
reciprocal: tensor([1.0000, 0.5000, 0.2500])
pow(2): tensor([ 1.,  4., 16.])
```

`x.sqrt()`, `x.exp()`, `x.log()` 일반적인 수학 연산을 텐서 메서드로 제공합니다.

예제:

```python
import torch
x = torch.tensor([1.0, 4.0, 9.0])
print(f"x: {x}")
print(f"sqrt: {x.sqrt()}")
print(f"exp: {torch.tensor([0.0, 1.0]).exp()}")
print(f"log: {torch.tensor([1.0, 2.718]).log()}")
```

실행 결과:

```
x: tensor([1., 4., 9.])
sqrt: tensor([1., 2., 3.])
exp: tensor([1.0000, 2.7183])
log: tensor([0.0000, 0.9999])
```

`x.item()  # get scalar value` 단일 요소 텐서에서 스칼라 값을 *Python* 숫자로 추출합니다.

예제:

```python
import torch
x = torch.tensor(3.14159)
val = x.item()
print(f"Tensor: {x}")
print(f"Python float: {val}")
print(f"Type: {type(val)}")
```

실행 결과:

```
Tensor: 3.141590118408203
Python float: 3.141590118408203
Type: <class 'float'>
```

`x.tolist()  # to Python list` 텐서를 중첩된 *Python* 리스트로 변환합니다.

예제:

```python
import torch
x = torch.tensor([[1, 2], [3, 4]])
lst = x.tolist()
print(f"Tensor: {x}")
print(f"List: {lst}")
print(f"Type: {type(lst)}")
```

실행 결과:

```
Tensor: tensor([[1, 2],
        [3, 4]])
List: [[1, 2], [3, 4]]
Type: <class 'list'>
```

`x.all(), x.any()  # boolean checks` 모든 또는 일부 요소가 *True* 인지 확인합니다.

예제:

```
import torch
x = torch.tensor([True, True, False])
print(f"x: {x}")
print(f"all: {x.all()}")
print(f"any: {x.any()}")
```

실행 결과:

```
x: tensor([ True,  True, False])
all: False
any: True
```

`x.nonzero()  # non-zero indices` *0* 이 아닌 요소의 인덱스를 반환합니다.

예제:

```
import torch
x = torch.tensor([0, 1, 0, 2, 0, 3])
indices = x.nonzero()
print(f"x: {x}")
print(f"Non-zero indices: {indices.squeeze()}")
```

실행 결과:

```
x: tensor([0, 1, 0, 2, 0, 3])
Non-zero indices: tensor([1, 3, 5])
```

`x.fill_(val), x.zero_()  # in-place` 값 또는 *0* 으로 제자리 채우기입니다. 언더스코어 접미사 = 제자리.

예제:

```
import torch
x = torch.empty(2, 3)
x.fill_(5.0)
print("After fill_(5.0):")
print(x)
x.zero_()
print("After zero_():")
print(x)
```

실행 결과:

```
After fill_(5.0):
tensor([[5., 5., 5.],
        [5., 5., 5.]])
After zero_():
tensor([[0., 0., 0.],
        [0., 0., 0.]])
```

`x.normal_(), x.uniform_()  # random` 제자리 랜덤 초기화입니다. 가중치 초기화에 유용합니다.

예제:

```python
import torch
torch.manual_seed(42)
x = torch.empty(2, 3)
x.normal_(mean=0, std=1)
print("After normal_(0, 1):")
print(x)
x.uniform_(0, 1)
print("After uniform_(0, 1):")
print(x)
```

실행 결과:

```
After normal_(0, 1):
tensor([[ 0.3367,  0.1288,  0.2345],
        [ 0.2303, -1.1229, -0.1863]])
After uniform_(0, 1):
tensor([[0.8694, 0.5677, 0.7411],
        [0.4294, 0.8854, 0.5739]])
```

`x.add_(y), x.mul_(y)  # in-place ops` 제자리 산술입니다. 텐서를 직접 수정하여 메모리를 절약합니다.

예제:

```python
import torch
x = torch.tensor([1.0, 2.0, 3.0])
print(f"Original: {x}")
x.add_(10)
print(f"After add_(10): {x}")
x.mul_(2)
print(f"After mul_(2): {x}")
```

실행 결과:

```
Original: tensor([1., 2., 3.])
After add_(10): tensor([11., 12., 13.])
After mul_(2): tensor([22., 24., 26.])
```